

# Flow Aplikasi Flutter uts\_live\_coding

Penjelasan alur aplikasi secara berurutan, dari aplikasi dijalankan sampai setiap fitur digunakan.

## 1. Flow Utama Aplikasi

1. **File pertama yang dijalankan:** `main.dart`, lewat fungsi `main()`.
2. **Class pertama yang dipanggil:** `main()` menjalankan `runApp(const MyApp())`, jadi class `MyApp`.
3. **Halaman pertama:** `MyApp` memakai `initialRoute: '/'`, dan route `'/'` mengarah ke `Homepage`. Jadi halaman pertama adalah **Homepage** (judul "Menu Navigator", dengan tombol Menu 1 dan Menu 2).
4. **Menu 1 ditekan:** `Navigator.push` membuka class `Page1` (di `homepage.dart`). `Page1` hanya mengembalikan `ProfilePage` (di `profile.dart`), sehingga halaman Profile tampil.
5. **Menu 2 ditekan:** `Navigator.push` membuka class `Page2` (di `kalkulator.dart`). `Page2` hanya mengembalikan `KalkulatorSederhana`, sehingga halaman kalkulator tampil.
6. **Kembali ke menu:**
  - Di Kalkulator: tombol "Back To Menu" memanggil `Navigator.pop(context)`.
  - Di Profile dan Kalkulator: tombol panah kembali di AppBar muncul otomatis dari Flutter karena halaman dibuka dengan `push`. Di `profile.dart` tidak ada kode tombol kembali sama sekali.
7. **Hubungan antar-file:** `main.dart` memanggil `homepage.dart`. `homepage.dart` memanggil `profile.dart` (Menu 1) dan `kalkulator.dart` (Menu 2).

## 2. Relasi Antar File

```
pubspec.yaml (konfigurasi proyek & aset)
  ↓ mendukung semuanya
main.dart
  ↓ memanggil (route '/')
homepage.dart
  |— Menu 1 ↓
  |   profile.dart
  |— Menu 2 ↓
      kalkulator.dart
```

| File                         | Fungsi dalam hubungan                                                    |
|------------------------------|--------------------------------------------------------------------------|
| <code>main.dart</code>       | Pintu masuk aplikasi, menyiapkan <code>MaterialApp</code> dan route awal |
| <code>homepage.dart</code>   | Halaman menu, penghubung ke Profile dan Kalkulator                       |
| <code>profile.dart</code>    | Halaman tujuan Menu 1 (kartu profil)                                     |
| <code>kalkulator.dart</code> | Halaman tujuan Menu 2 (penjumlahan)                                      |
| <code>pubspec.yaml</code>    | Konfigurasi proyek, termasuk daftar aset gambar                          |

### 3. Flow Per File

#### 1) `main.dart`

**Fungsi:** Titik awal aplikasi.

**Flow:** `main()` menjalankan `MyApp`, lalu `MaterialApp` membuka route `'/'` yaitu `Homepage`.

**Baris kode penting:**

- `runApp(const MyApp());` → menjalankan aplikasi dengan `MyApp` sebagai widget utama.
- `initialRoute: '/',` → menentukan halaman pertama yang dibuka.
- `'/': (context) => const Homepage(),` → route `'/'` diarahkan ke `Homepage`.

**Catatan jujur dari source code:**

- Route `'/profile'` dan `'/kalkulator'` sudah didaftarkan, tetapi **tidak dipakai**, karena navigasi di `homepage.dart` memakai `MaterialPageRoute`, bukan `pushNamed`.
- Class `FirstRoute`, `SecondRoute`, dan `ThirdRoute` tidak dipanggil di mana pun, jadi tidak berpengaruh pada flow.
- File `main.dart` yang diunggah **terpotong** di `ThirdRoute` (method `build` tidak selesai, tidak ada kurung penutup). Kode ini akan **error saat di-compile** kalau memang seperti itu. Pastikan file aslinya lengkap, atau hapus class yang tidak terpakai.

#### 2) `homepage.dart`

**Fungsi:** Halaman menu untuk memilih tujuan.

**Flow:** Menampilkan dua tombol. Tiap tombol membuka halaman baru dengan `Navigator.push`.

**Baris kode penting:**

- `Navigator.push(context, MaterialPageRoute(builder: (context) => const Page1()));` → pindah ke halaman Menu 1 (Page1 → ProfilePage).
- `MaterialPageRoute(builder: (context) => const Page2())` → pindah ke halaman Menu 2 (Page2 → KalkulatorSederhana).
- `class Page1 ... return const ProfilePage();` → Page1 hanya perantara ke ProfilePage.
- `import 'kalkulator.dart'; import 'profile.dart';` → menghubungkan file ini ke dua halaman tujuan.

### 3) profile.dart

**Fungsi:** Menampilkan kartu profil (Gea Dwi Putri).

**Flow:** ProfilePage (Scaffold + AppBar) menampilkan ProfilCard di tengah layar. ProfilCard berisi foto, nama, deskripsi, dan kontak (memakai KontakInfoRow). Halaman ini hanya menampilkan data, tanpa logika dan tanpa tombol navigasi sendiri.

**Baris kode penting:**

- `body: const Center(child: ProfilCard()),` → menampilkan kartu profil.
- `AssetImage('images/gea.jpg')` → mengambil foto dari folder aset (didaftarkan di pubspec.yaml).
- `KontakInfoRow(icon: ..., teks: ...)` → komponen berulang untuk email, telepon, dan lokasi.

### 4) kalkulator.dart

**Fungsi:** Kalkulator penjumlahan dua angka.

**Flow:** Page2 memanggil KalkulatorSederhana, yaitu StatefulWidget yang menerima input, menghitung, dan menampilkan hasil (dijelaskan di bagian 4).

**Baris kode penting:**

- `class Page2 ... return const KalkulatorSederhana();` → perantara dari Menu 2.
- `class KalkulatorSederhana extends StatefulWidget` → stateful karena hasil bisa berubah.

## 4. Flow Kalkulator

```

Input Angka 1
↓
Input Angka 2
↓
Tekan "Proses Hitung"
↓
Validasi input
↓
Mengambil nilai angka
↓
Melakukan penjumlahan
↓
Menyimpan hasil dengan setState
↓
Menampilkan hasil
↓
Tekan "Back To Menu"
↓
Kembali ke halaman sebelumnya

```

- 1. Input Angka 1 dan 2:** `TextFormField(controller: _angka1Controller, keyboardType: TextInputType.number, ...)` dan `TextFormField(controller: _angka2Controller, ...)`. Controller menyimpan teks yang diketik.
- 2. Tekan "Proses Hitung":** `onPressed: _hitung`, memanggil fungsi `_hitung()`.
- 3. Validasi input:** `if (_formKey.currentState!.validate())` memeriksa semua validator. Kalau kolom kosong, muncul pesan "Masukkan angka pertama/dua" dan perhitungan berhenti.
- 4. Mengambil nilai angka:** `double angka1 = double.parse(_angka1Controller.text);` dan `double angka2 = double.parse(_angka2Controller.text);` mengubah teks menjadi angka desimal.
- 5. Penjumlahan:** `double total = angka1 + angka2;`
- 6. Menyimpan hasil dengan setState:** `setState(() { hasil = total.toString(); });` mengisi variabel `hasil` dan memberi tahu Flutter agar layar digambar ulang.
- 7. Menampilkan hasil:** `Text('Hasil penjumlahan: $hasil', ...)` menampilkan nilai `hasil` terbaru.
- 8. Tekan "Back To Menu":** `Navigator.pop(context);` menutup halaman kalkulator dan kembali ke `Homepage`.

**Catatan:** Validator hanya memeriksa kolom kosong. Input seperti `-` atau `1.2.3` lolos validasi tetapi akan membuat `double.parse` error. Ini bisa disebut sebagai kekurangan kalau ditanya dosen.

## 5. Fungsi `pubspec.yaml`

`pubspec.yaml` adalah file konfigurasi proyek. Ia tidak dipanggil dari kode Dart, tetapi mendukung seluruh aplikasi:

- `name: uts_live_coding` → nama proyek.
- `dependencies: flutter: sdk: flutter` → library Flutter yang dipakai semua file (`package:flutter/material.dart`).
- `cupertino_icons` → paket ikon bergaya iOS.
- `uses-material-design: true` → mengaktifkan ikon Material, dipakai di `profile.dart` (`Icons.email`, `Icons.phone`, `Icons.location_on`).
- `assets: - images/` → mendaftarkan folder `images/`. Ini **penting** untuk `profile.dart`, karena `AssetImage('images/gea.jpg')` hanya bisa tampil jika folder tersebut terdaftar di sini.
- `environment: sdk: ^3.13.2` → versi Dart SDK yang dibutuhkan.

**Catatan:** File `gea.jpg` tidak ada di unggahan, jadi tidak bisa dipastikan gambarnya ada di folder `images/`. Kalau tidak ada, halaman Profile akan error saat memuat gambar.

## 6. Kesimpulan Flow

Aplikasi dimulai dari `main.dart`, yang menjalankan `MyApp` dan membuka `Homepage` sebagai halaman menu. Dari `Homepage`, **Menu 1** membuka `profile.dart` (lewat `Page1`) dan **Menu 2** membuka `kalkulator.dart` (lewat `Page2`). Perpindahan memakai `Navigator.push`, dan kembali ke menu memakai `Navigator.pop` atau tombol panah di AppBar. `pubspec.yaml` menyediakan dasar proyek, yaitu library Flutter, ikon, dan aset gambar yang dipakai halaman Profile.

Satu kalimat untuk presentasi: *"main membuka Homepage, Homepage menghubungkan ke Profile dan Kalkulator dengan Navigator.push, lalu kembali dengan Navigator.pop, dan pubspec mendukung dengan mendaftarkan library dan aset."*